5

Solving Problems By Searching

Chapter Learning Outcomes

CLO

5

- Define a search problem.
- Compute the size of a search tree.
- Compute the branch factor of a search tree.
- Know the branching factors for some games.
- Compute the number of nodes in a search tree.
- Distinguish between maximum depth and goal depth.
- Approximate a time to search for a goal in a search tree.
- Approximate a space when searching for a goal in a search tree.
- Know different search strategies.
- Understand the criteria for evaluating search strategies.
- Understand, apply and evaluate (BFS, DFS, DLS, and IDS) algorithms.

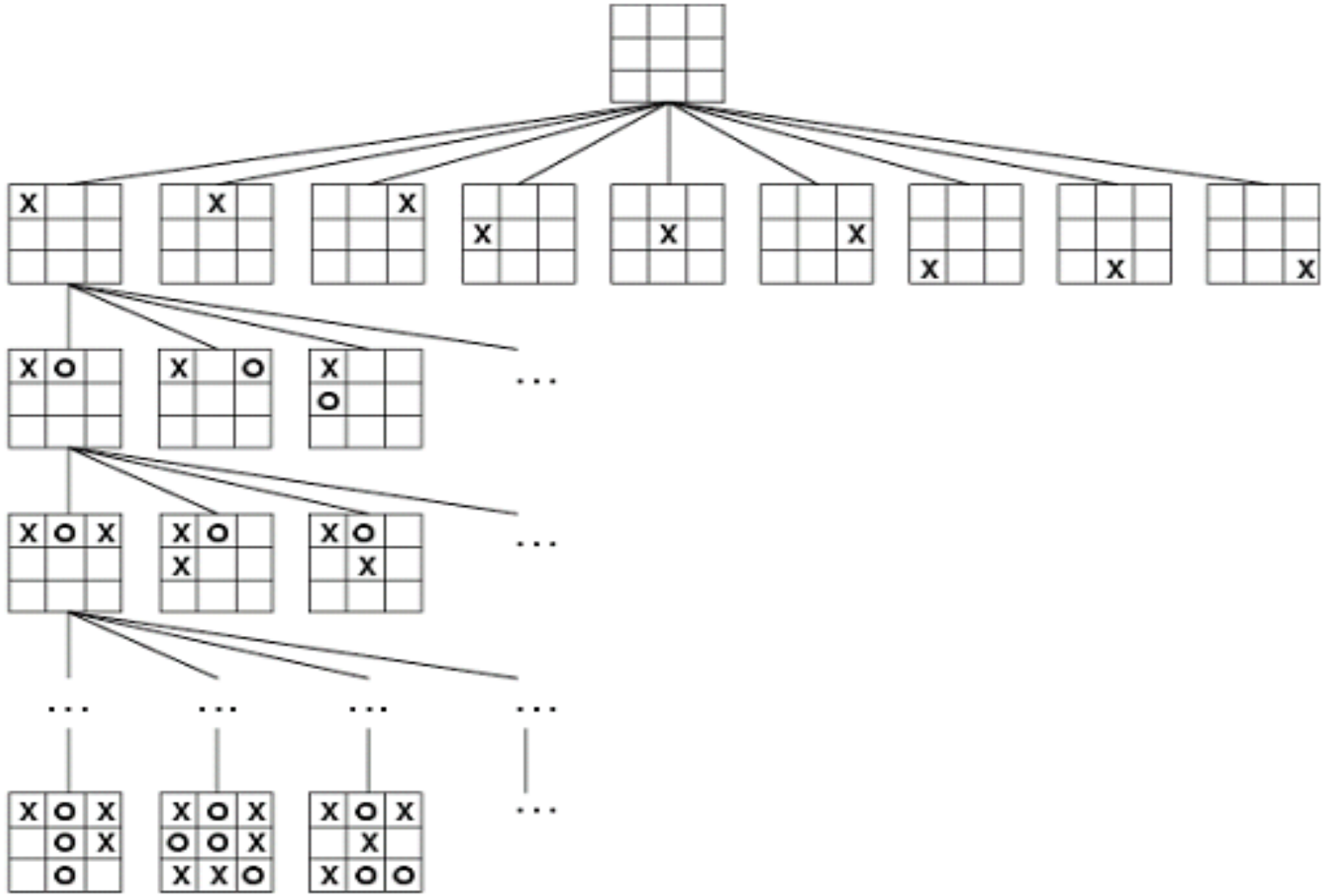
Defining a Search Problem

Search is one of the most powerful approaches to problem solving in AI. A search problem is defined by four items:

- **Initial state**
- **Actions (Operators):** operators are legal actions which the agent can take to move from one state to another.
- **Goal test:** Determine if the current state is the goal state or not.
- **Path cost:**
  - e.g., sum of distances, number of actions executed, etc.

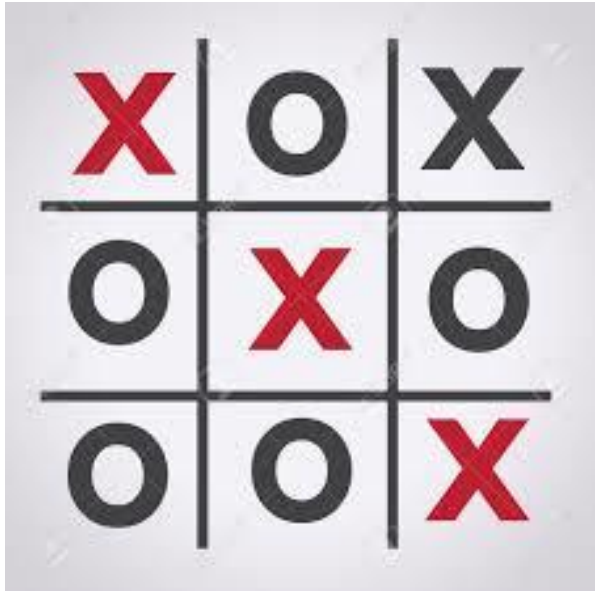
A solution is a sequence of actions leading from the initial state to a goal state

A Search Tree for Tic-Tac-Toe



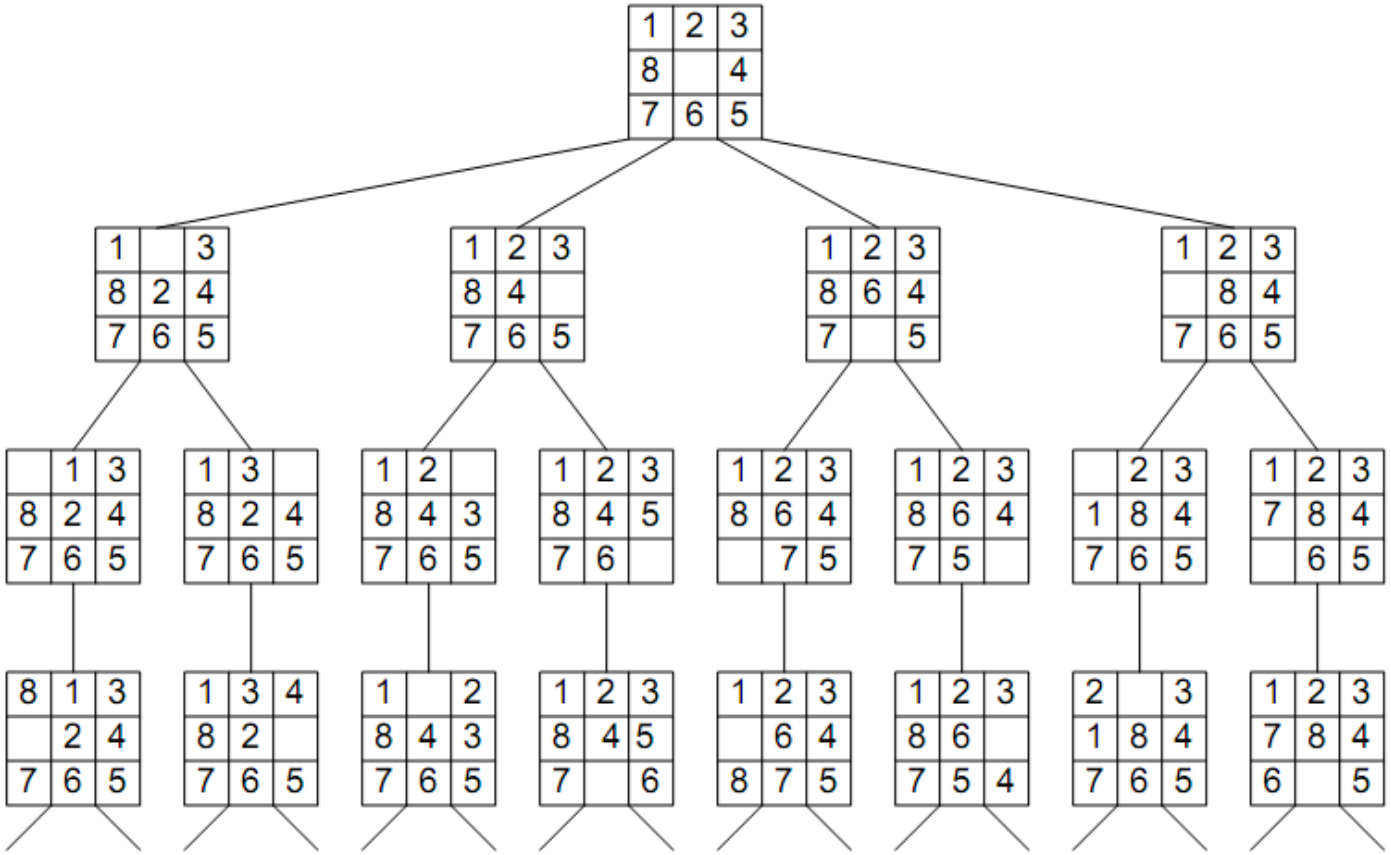
Quiz

Define the tic-tac-toe game as a search problem in AI.



د. حسن المعيدي

A Search Tree for 8-Puzzle



Typically AI problems will have a very large space.

What is the size of a Search Tree?

8 - Puzzle

2	1	3
4	7	6
5	8	

Number of Nodes = 6,964,551

How did we compute this value?

د. حسن المعيدي

What is the size of a Search Tree?

Rubik's Cube

Number of Nodes = 193,484,242,857,296,000,000

How did we compute this value?

What is the size of a Search Tree?

Chess

Number of Nodes =

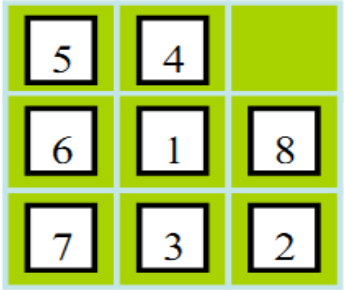
7,833,394,781,046,530,000,000,000,000,000

How did we compute this value?

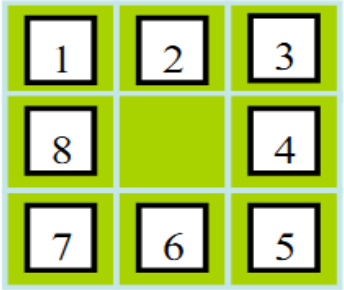
Defining a Search Problem

EXAMPLE

Define the 8-Puzzle game as a search problem in AI.



initial state



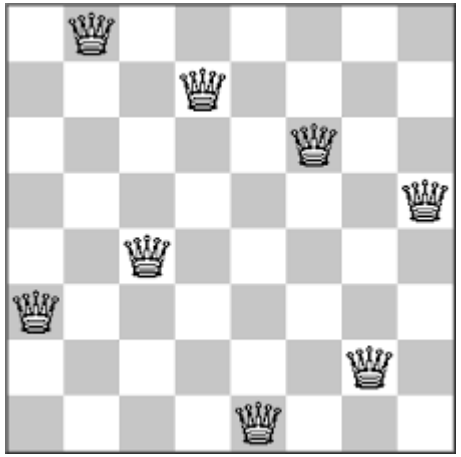
goal state

د. حسن المعيدي

Defining a Search Problem

EXAMPLE

Define the 8-queens game as a search problem in AI.



- **Initial State:** No queens on the board
- **Actions** : add a queen to any square.
- **Goal Test** : 8 queens on board, none attacked.
- **Path cost** : 1 per move

د. حسن المعيدي

- Search Tree.
- Branch Factor.
- Number of Sates.

Search Tree

- Each node in a search tree has child nodes that represent each of the states reachable by the parent state.

- Another way of looking at it: child nodes represent all available options once the search procedure reaches the parent node.

- There are a number of strategies for traversing a search tree.

د. حسن المعيدي

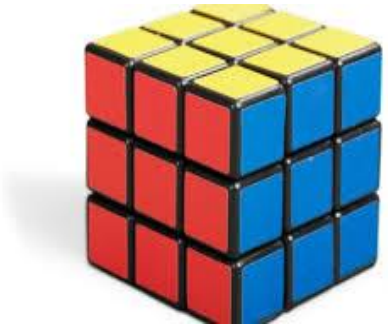
What is the size of a Search Tree?

Number of Nodes = 6,964,551

2	1	3
4	7	6
5	8	

How did we compute these values?

r of Nodes = 193,484,242,857,296,000,000



Number of Nodes =
7,833,394,781,046,530,000,000,000,000,000

د. حسن المعيدي

Branch Factor

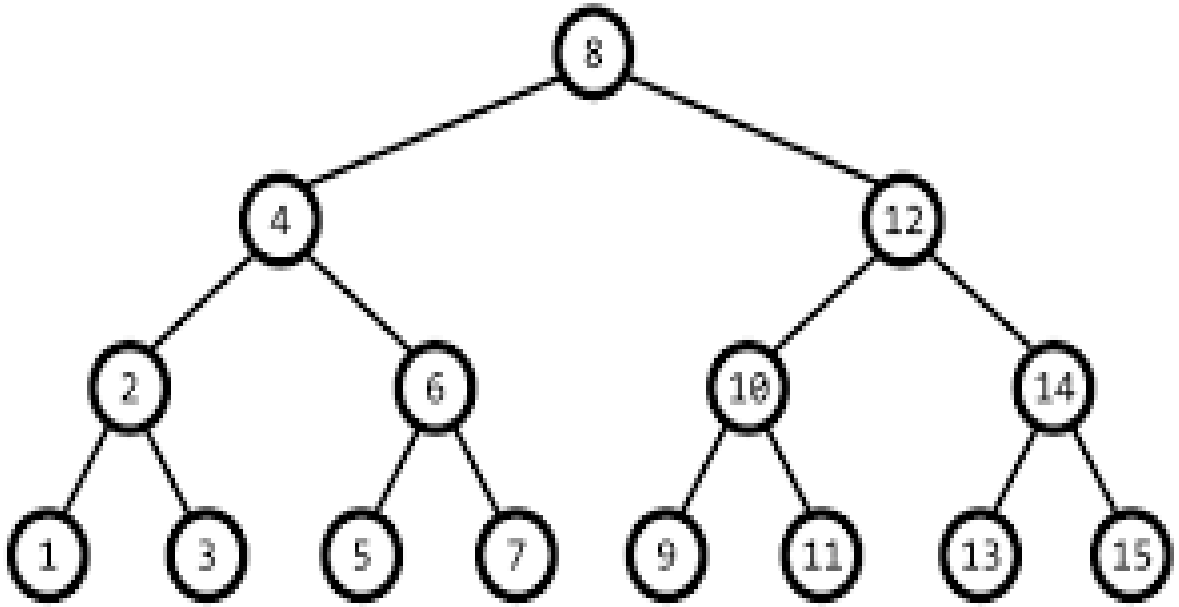
In computing, tree data structures, and game theory, the **branching factor (BF)** is the number of children at each node. If the branching factor is not uniform, an average branching factor can be calculated.

د. حسن المعيدي

Branch Factor

EXAMPLE

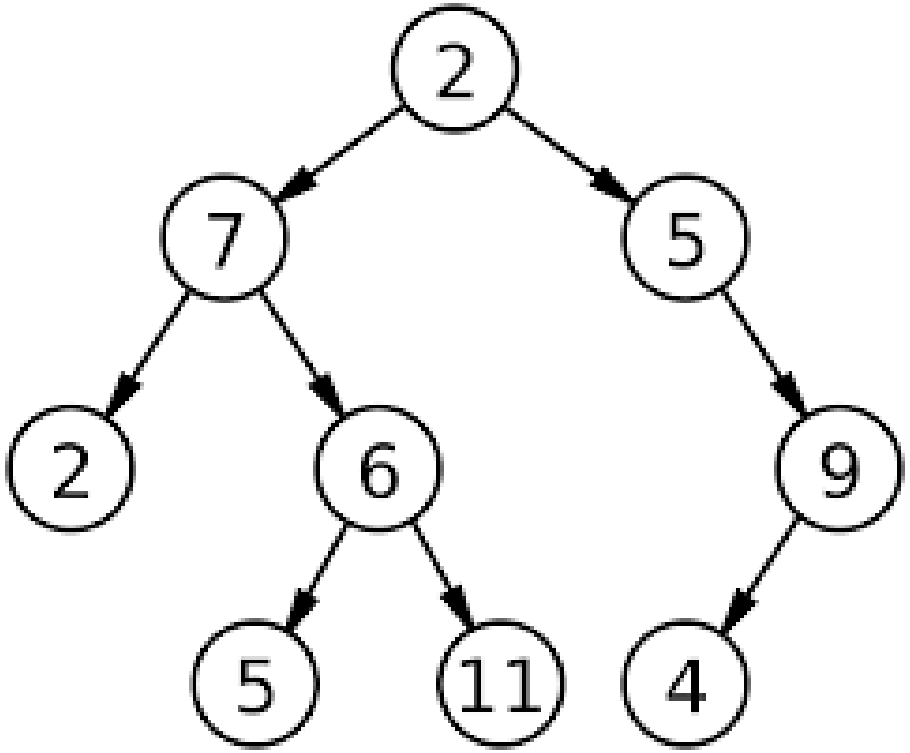
Compute the branch factor (BF) for the following tree:



Branch Factor

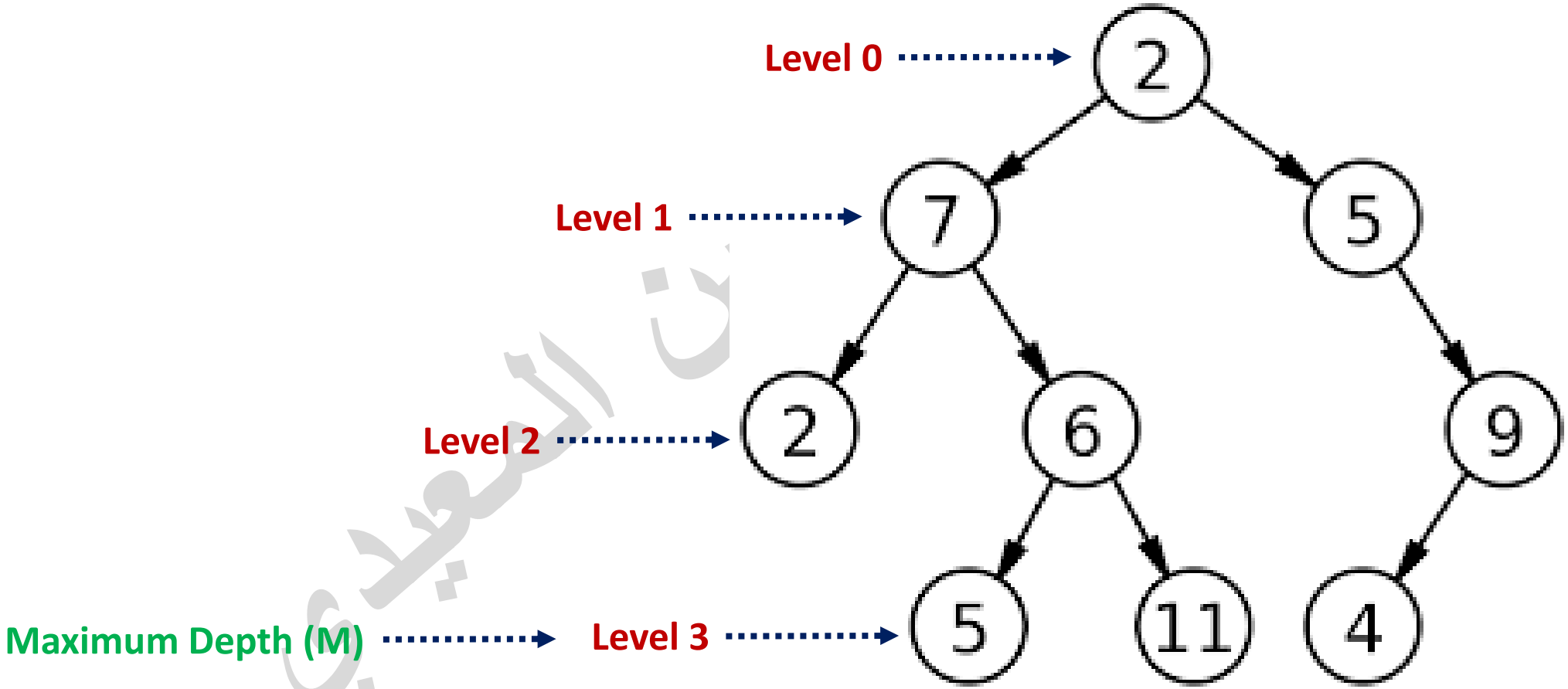
EXAMPLE

Compute the branch factor (BF) for the following tree:



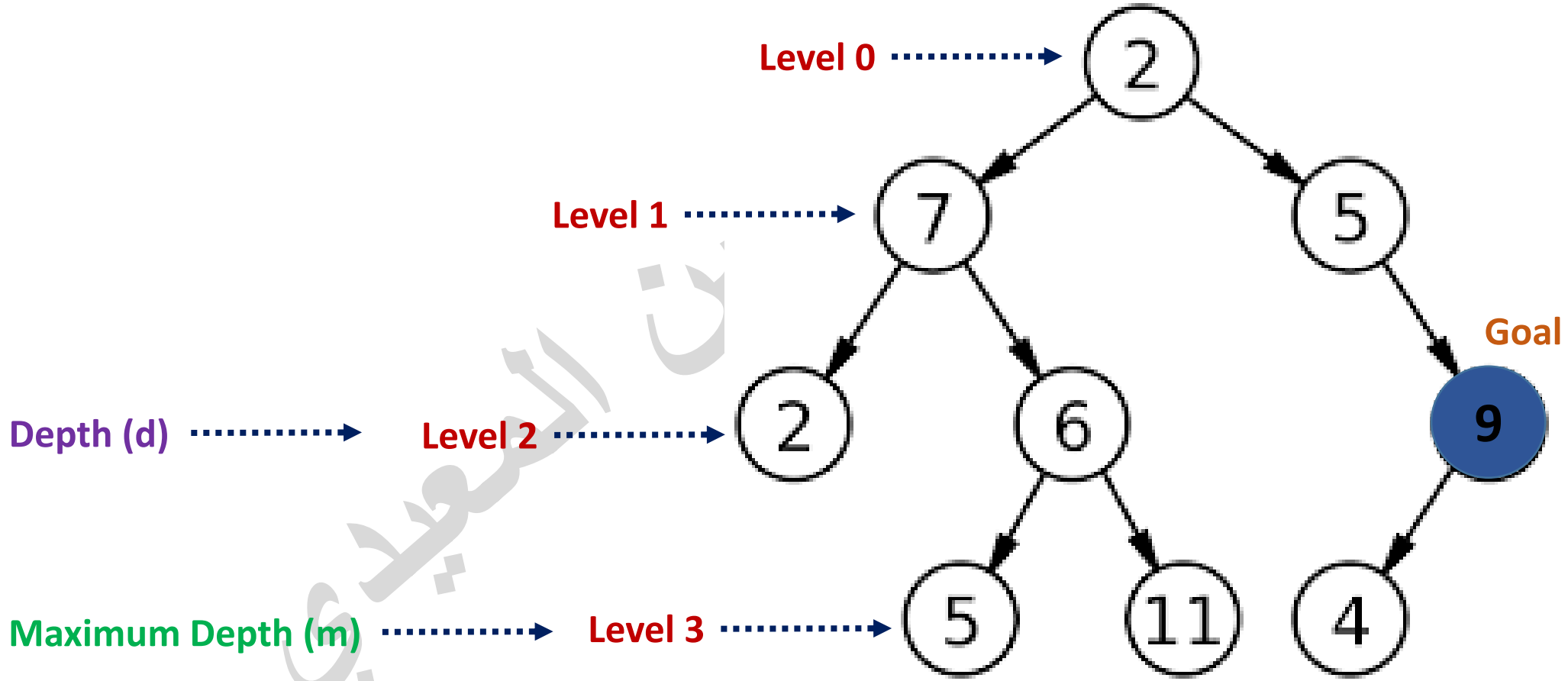
د. حسن المعيدي

Maximum Depth of a State-Space (Tree)



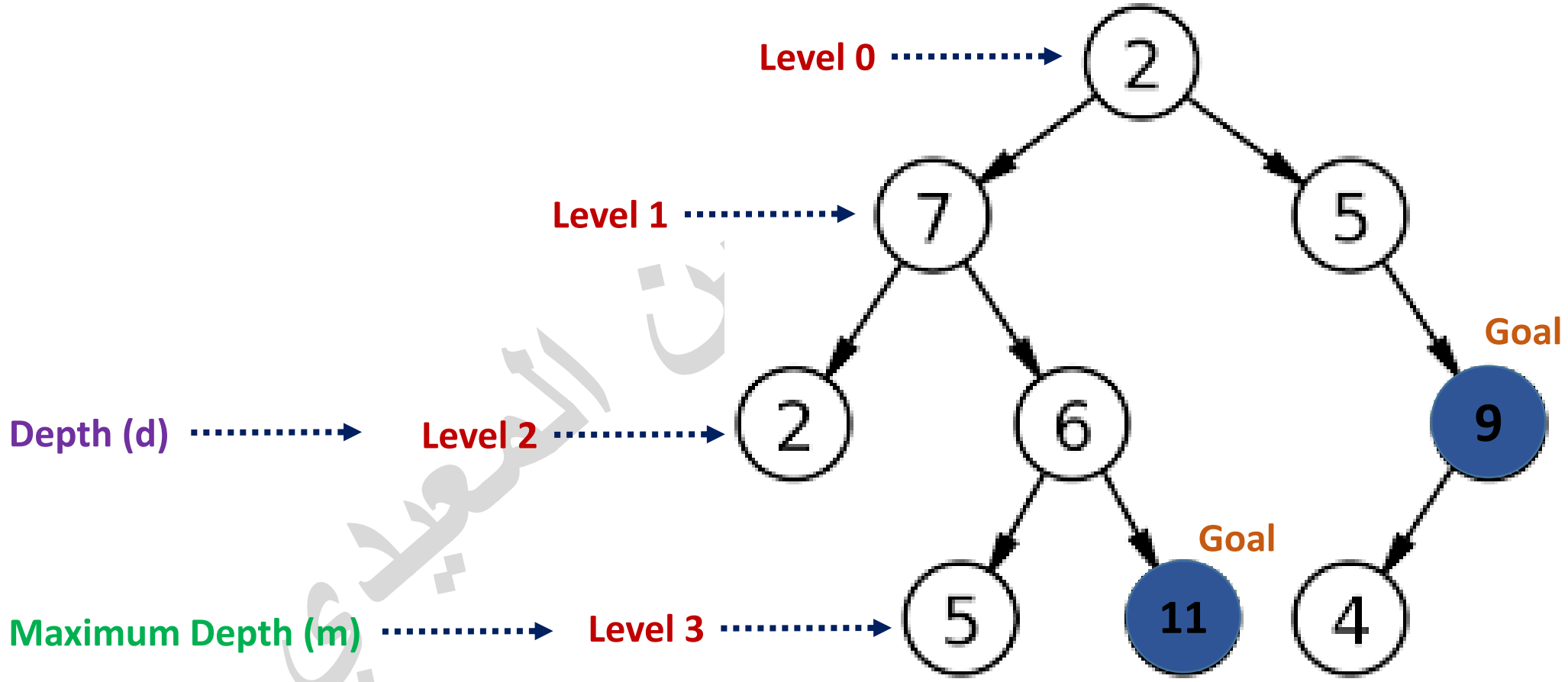
m=3

Depth of the Optimal Solution in a State-Space (Tree)



m=3 d=2

Depth of the Optimal Solution in a State-Space (Tree)



m=3 d=2

How to Find the Number of Nodes in a Search Tree?

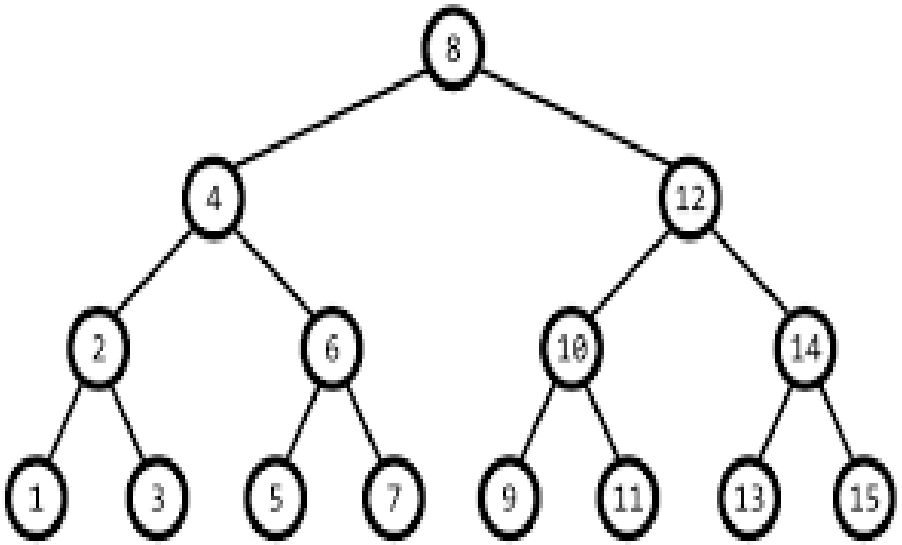
د. حسن المعيدي

Example

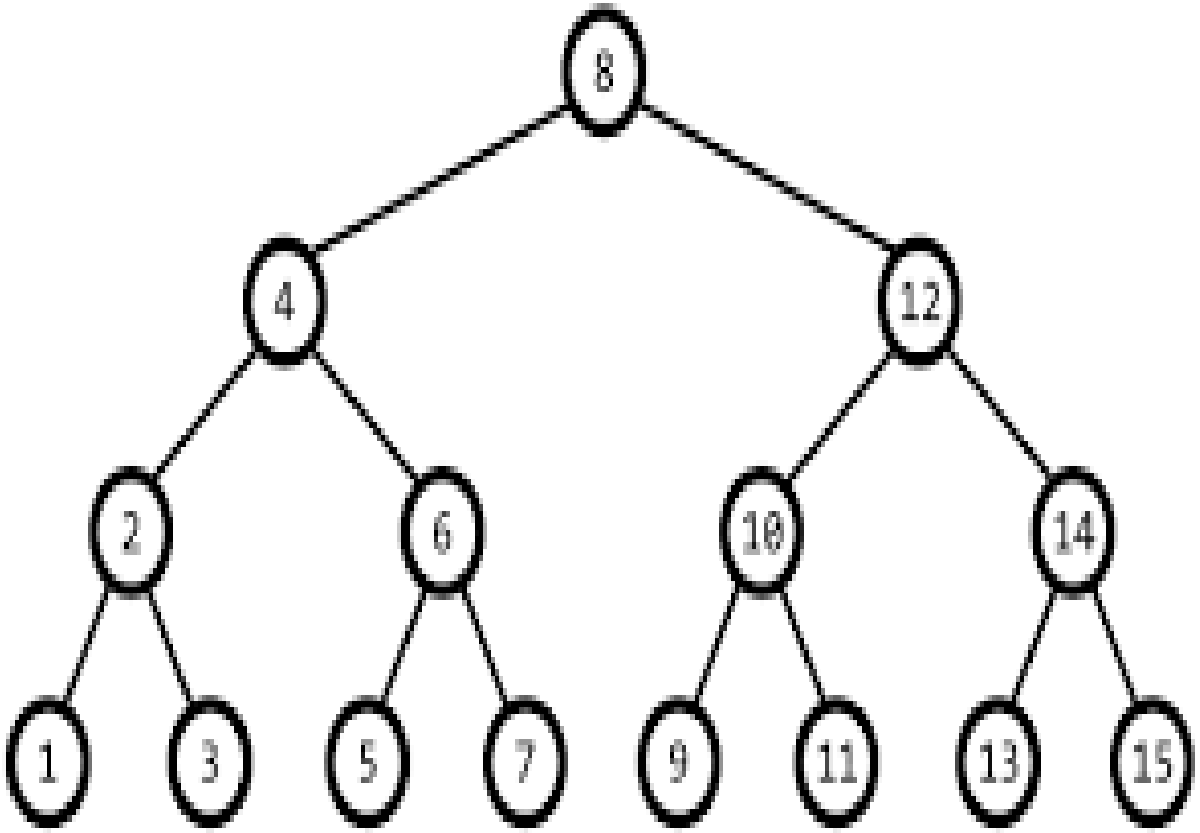
EXAMPLE

Assume the goal state is (15), find:

1. Branch Factor (BF).
2. Maximum Depth (m).
3. Goal Depth (d).
4. Number of Nodes (n).



Example - Solution



المعيدي

Branching Factors for some Games

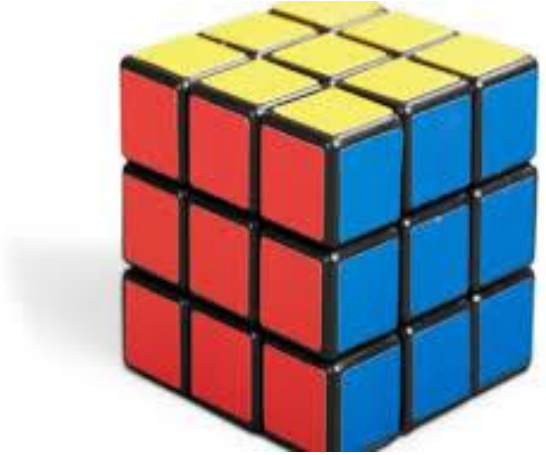
8 - Puzzle

2	1	3
4	7	6
5	8	

د. حسن المعيدي

Branching Factors for some Games

Rubik's Cube



د. حسن المعيدي

Branching Factors for some Games

Chess

د. حسن المعيدي

Example

EXAMPLE

Assume you can process 300 states per second to search a given game tree using a blind search. What is the approximate time to search for a goal, assuming the branching factor (BF)=5 and the goal is in level 7?

د. حسن المعيدي

Search Strategies

Un-informed (Blind) Search

- Breadth-First-Search (BFS)
- Depth-First-Search (DFS)
- Depth Limited Search (DLS)
- Iterative Deepening (IDS)
- Bidirectional Search (BS)

Informed (Heuristic) Search

- Greedy Best-First-Search (GBFS)
- A*
- IDA*
- SMA*
- Hill Climbing
- Beam search

Optimization Search

- Hill climbing
- Simulated Annealing
- Genetic Algorithms
- Ant Colony

Game playing

- Minimax Algorithm
- Alpha-Beta Pruning

د. حسن المعيدي

Criteria for Evaluating Search Strategies

Completeness	Is the strategy guaranteed to find a solution when there is one?
Optimality	Does the strategy find the highest-quality solution when there are several solutions?
Time Complexity	How long does it take to find a solution?
Space Complexity	How much memory does it need to perform the search?

د. حسن المعيدي

Breadth-First Search

(BFS)

Breadth-First Search (BFS)

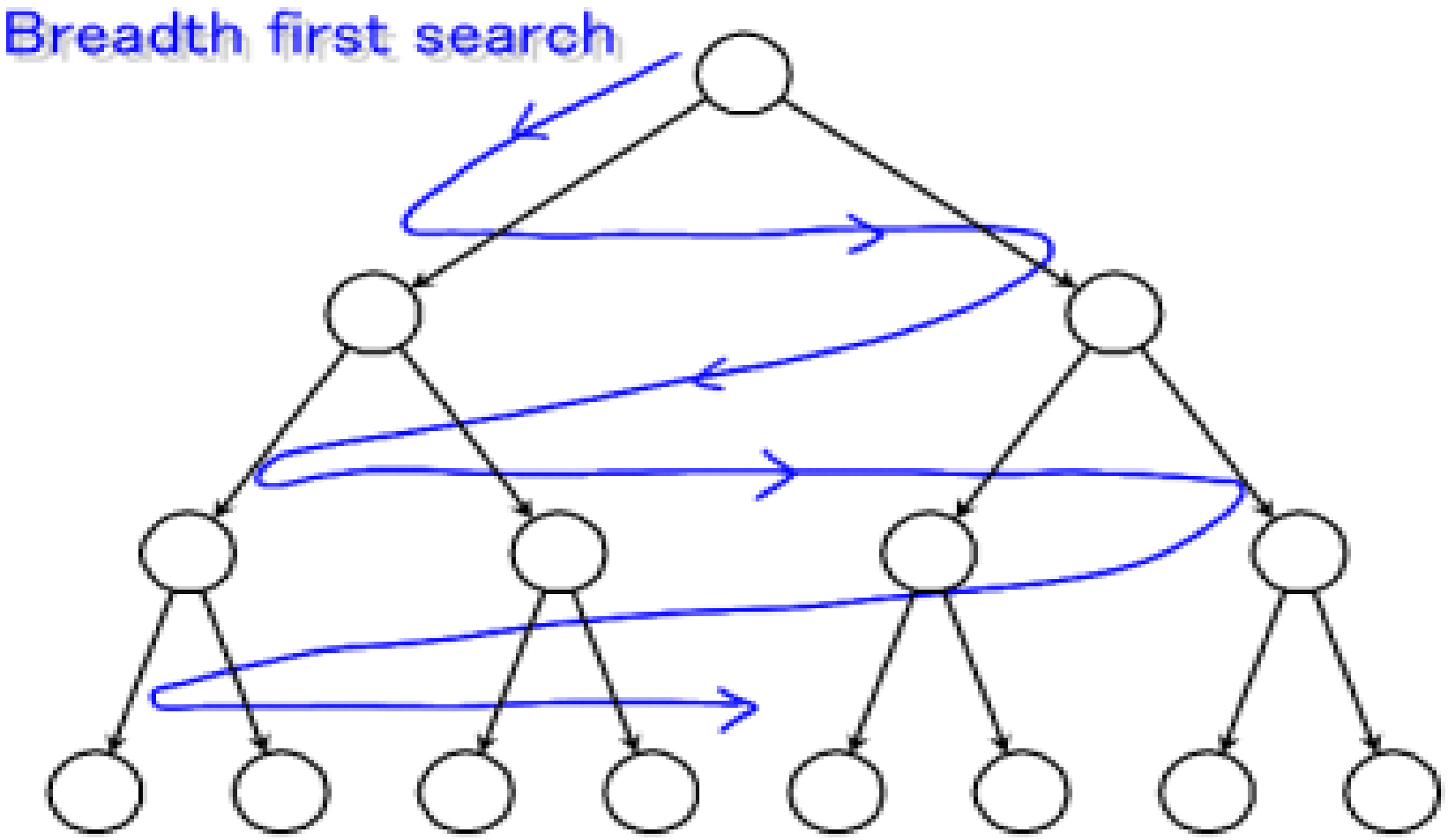
- BFS belongs to uninformed (blind) strategies (algorithms).

- Uninformed (blind) strategies use only the information available in the problem definition. These strategies order nodes without using any domain specific information.

- The following are examples for uninformed (blind) strategies.
  - **Breadth-First-Search (BFS)**
  - **Depth-First-Search (DFS)**
  - **Depth Limited Search (DLS)**
  - **Iterative Deepening (IDS)**
  - **Bidirectional Search (BS)**

د. حسن المعيدي

Breadth-First Search (BFS)



Breadth-First Search (BFS)

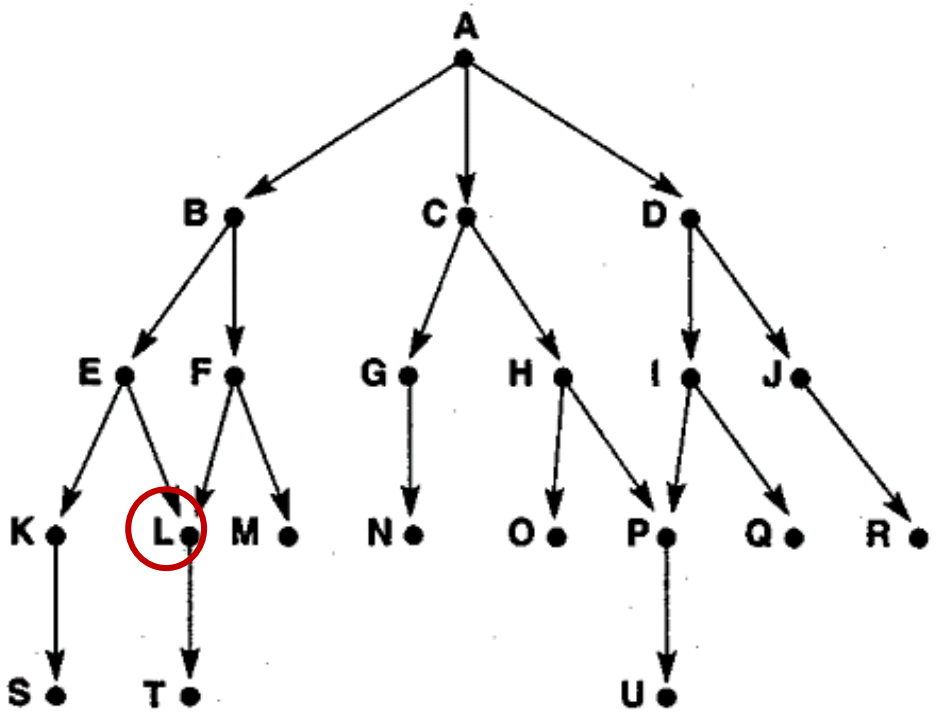
```
function breadth_first_search;

begin
   open := [Start];                                                    % initialize
   closed := [ ];
   while open ≠ [ ] do                                                 % states remain
      begin
         remove leftmost state from open, call it X;
            if X is a goal then return SUCCESS                         % goal found
               else begin
                  generate children of X;
                  put X on closed;
                  discard children of X if already on open or closed;  % loop check
                  put remaining children on right end of open          % queue
               end
      end
   return FAIL                                                         % no states left
end.
```

Breadth-First Search (BFS) - Example

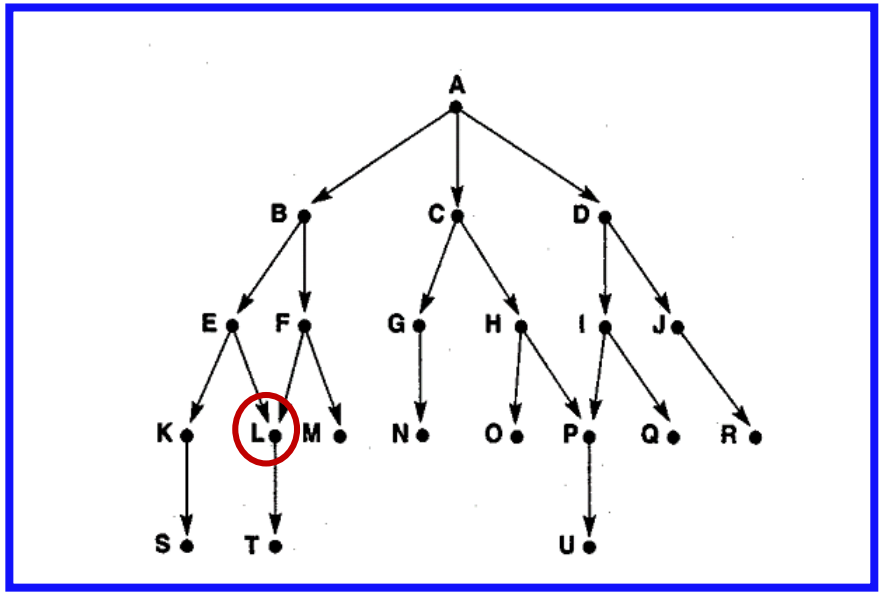
EXAMPLE

Apply the BFS algorithm for the following graph.



Breadth-First Search (BFS)

i	[OPEN]	X
1		
2		
3		
4		
5		
6		
7		
8		
9		
10		
11		
12		



د. حسن المعيدي

Breadth-First Search (BFS)

1- What is the content of OPEN array at iteration 3?

2- What is the value of (x) at iteration 3?

3- What is the first item of OPEN array at iteration 3?

4- What is the second item of OPEN array at iteration 3?

د. حسن المعيدي

Breadth-First Search (BFS)

5- What is the content of OPEN array at iteration 7?

6- What is the value of (x) at iteration 5?

7- What is the order of the expanded states?

8- What is the solution path?

د. حسن المعيدي

Evaluating the Breadth-First Search (BFS)

- One of the simplest search strategy.
- Time and Space are complex.
- Cannot be use to solve any but it can use to solve the smallest problems. See Slide (338) for a simulation.

Completeness — **Is the strategy guaranteed to find a solution when there is one?**

Optimality — **Does the strategy find the highest-quality solution when there are several solutions?**

د. حسن المعيدي

Evaluating the Breadth-First Search (BFS)

Time Complexity How long does it take to find a solution?

د. حسن المعيدي

Evaluating the Breadth-First Search (BFS)

Space Complexity

How much memory does it need to perform the search?

د. حسن المعيدي

Examples of Time and Memory Requirements for (BFS)

Depth of Solution	Nodes Expanded	Time	Memory
0	1	1 millisecond	100 bytes
2	111	0.1 seconds	11 kbytes
4	11,111	11 seconds	1 megabyte
8	10^8	31 hours	11 gigabytes
12	10^{12}	35 years	111 terabytes

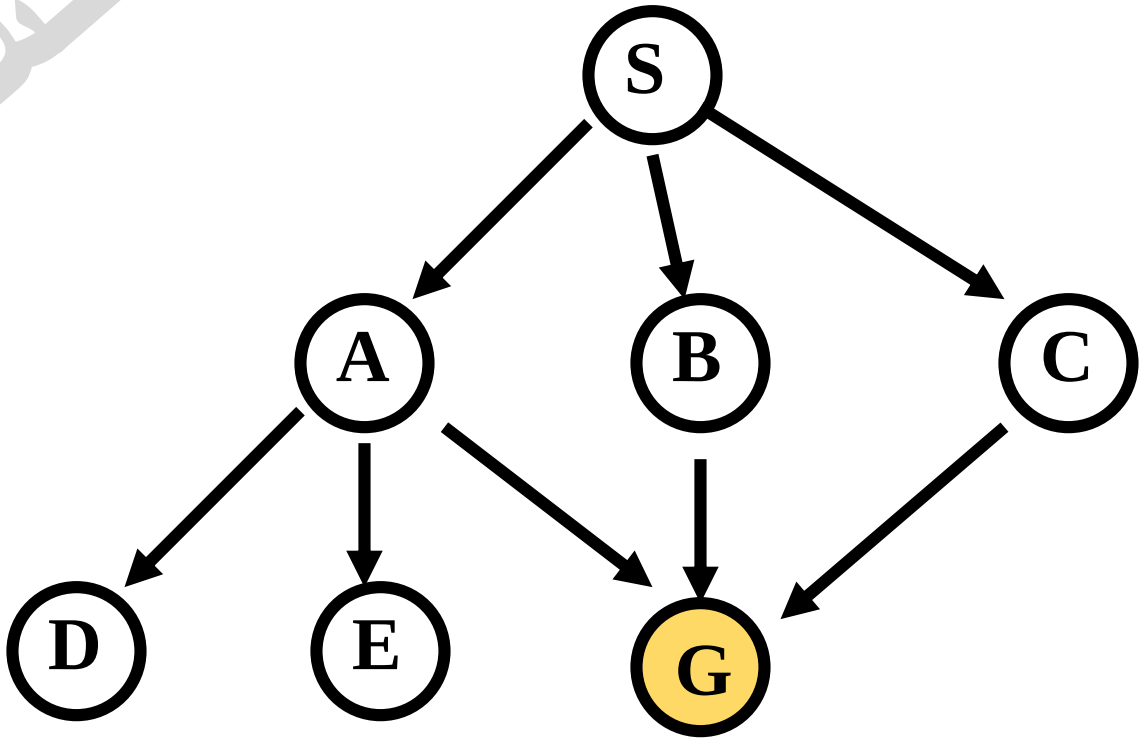
Assuming:

- BF=10
- 1000 nodes/sec
- 100 bytes/node

د. حسن المعيدي

Quiz

Apply the BFS algorithm for the following graph.

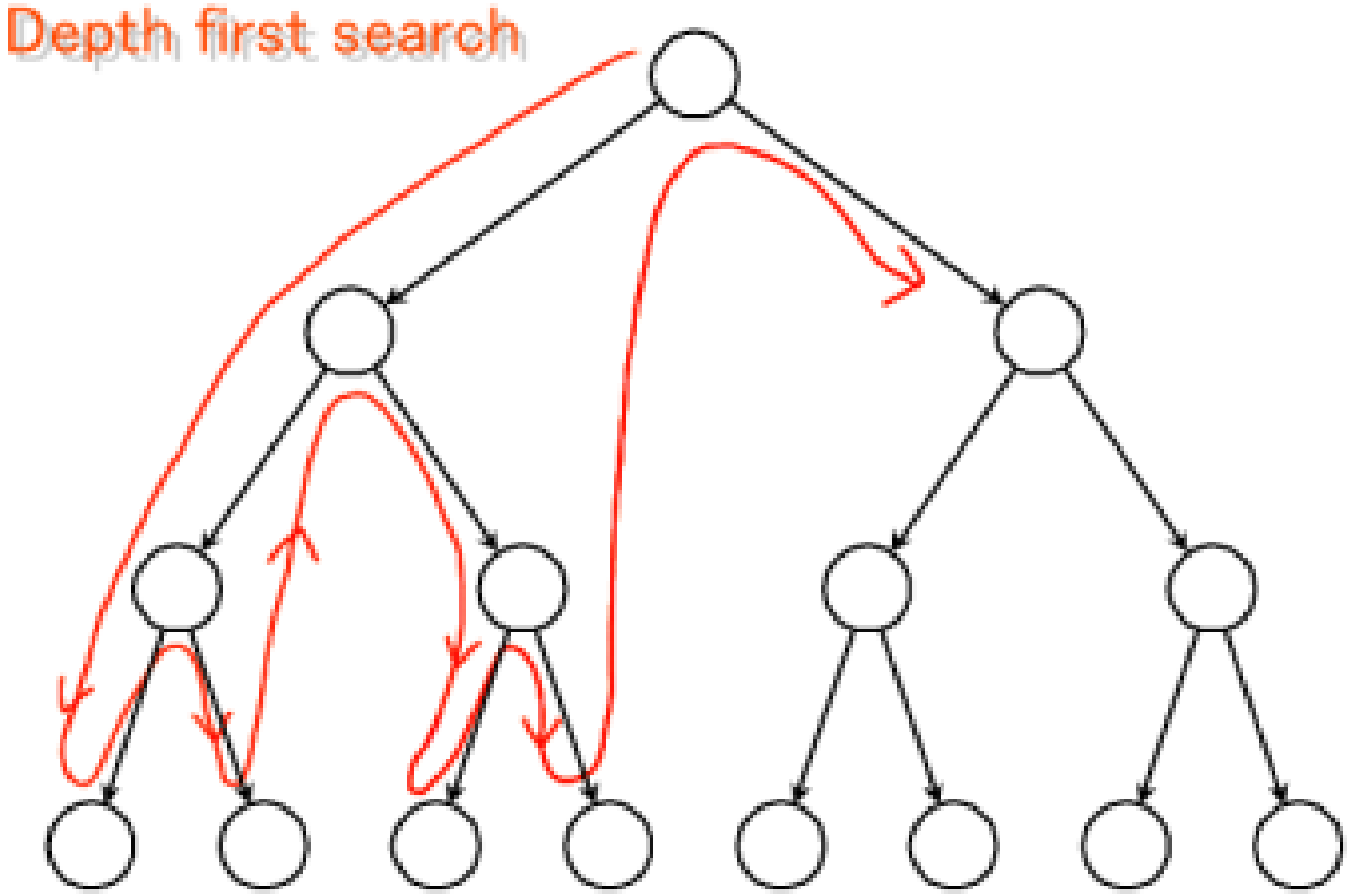


حسن المعيدي

Depth-First Search

(DFS)

Depth-First Search (DFS)



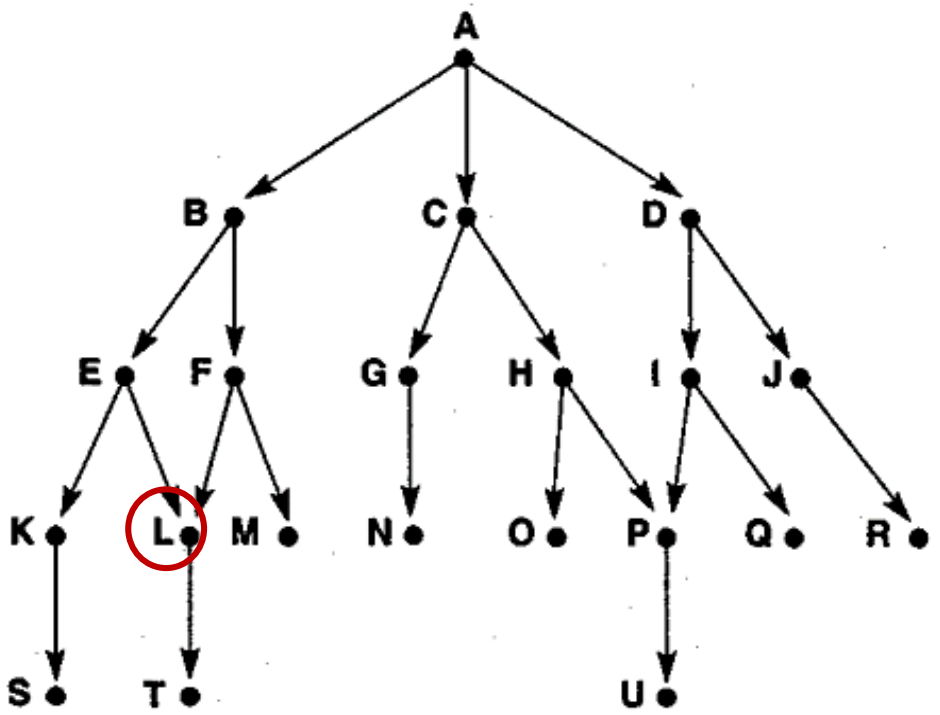
Depth-First Search (DFS)

```
begin
   open := [Start];                                                    % initialize
   closed := [ ];
   while open ≠ [ ] do                                                 % states remain
      begin
         remove leftmost state from open, call it X;
         if X is a goal then return SUCCESS                            % goal found
            else begin
               generate children of X;
               put X on closed;
               discard children of X if already on open or closed;     % loop check
               put remaining children on left end of open              % stack
            end
      end;
   return FAIL                                                         % no states left
end.
```

Depth-First Search (DFS) - Example

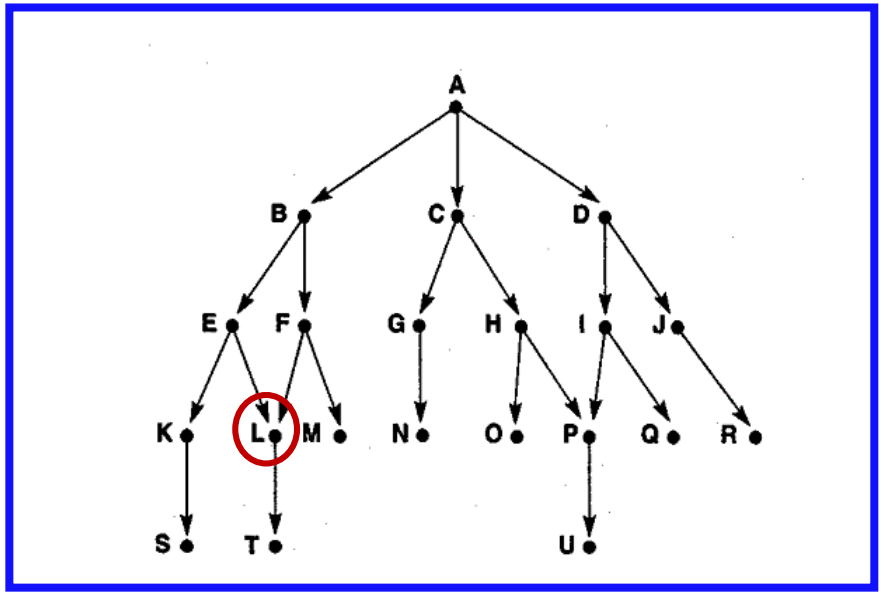
EXAMPLE

Apply the DFS algorithm for the following graph.



Depth-First Search (DFS)

i	[OPEN]	X
1		
2		
3		
4		
5		
6		
7		
8		
9		
10		
11		



د. حسن المعيدي

Depth-First Search (DFS)

1- What is the content of OPEN array at iteration 3?

2- What is the value of (x) at iteration 3?

3- What is the first item of OPEN array at iteration 3?

4- What is the second item of OPEN array at iteration 3?

د. حسن المعيدي

Depth-First Search (DFS)

5- What is the content of OPEN array at iteration 4?

6- What is the value of (x) at iteration 5?

7- What is the order of the expanded states?

8- What is the solution path?

د. حسن المعيدي

Evaluating the Depth-First Search (DFS)

- One problem with DFS is that it can get stuck going down the wrong path.
- Time and Space are complex.
- Many problems have **very deep** or even **infinite** search trees.
- DFS should be **avoided** for search trees with **large or infinite maximum depths**.

Completeness — **Is the strategy guaranteed to find a solution when there is one?**

Optimality — **Does the strategy find the highest-quality solution when there are several solutions?**

د. حسن المعيدي

Evaluating the Depth-First Search (DFS)

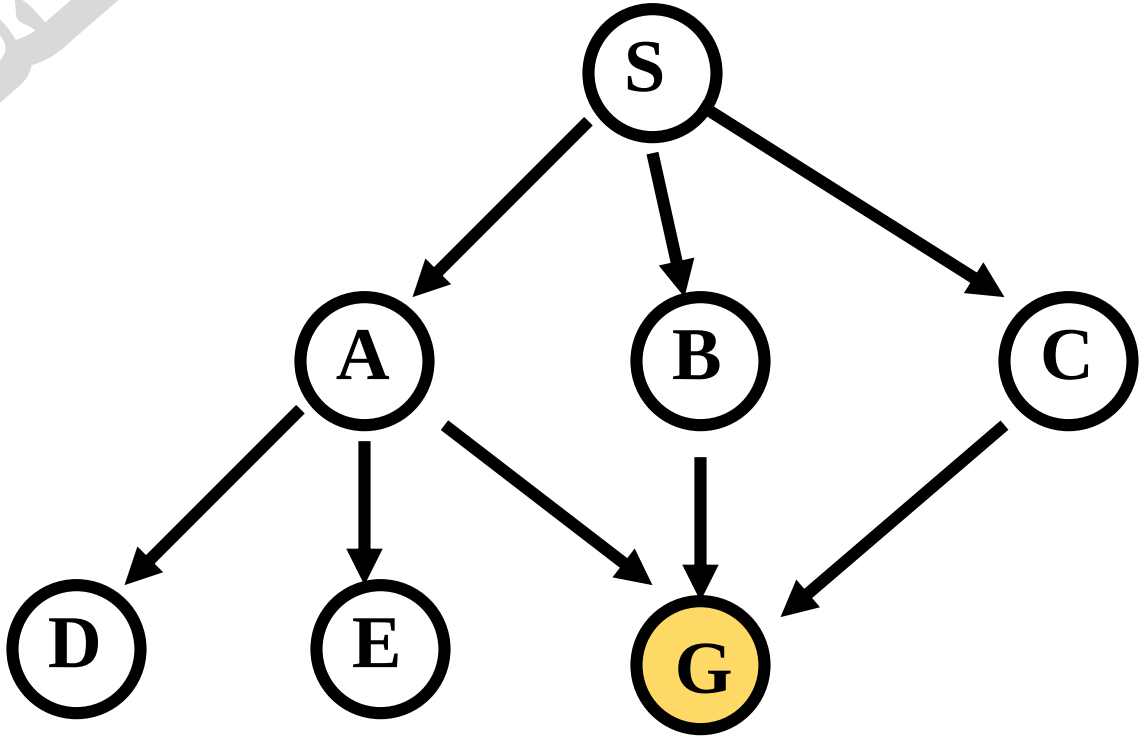
Time Complexity How long does it take to find a solution?

Space Complexity How much memory does it need to perform the search?

د. حسن المعيدي

Quiz

Apply the DFS algorithm for the following graph.



حسن المعيدي

Depth-Limited Search

(DLS)

Depth Limited Search (DLS)

- **Stop DFS at a *fixed* depth *f*.**

- **Goal, may not be found: *Incomplete Algorithm***
  - **If goal depth *d* > *f*.**

- **Why DLS?**
-
- **Time:**

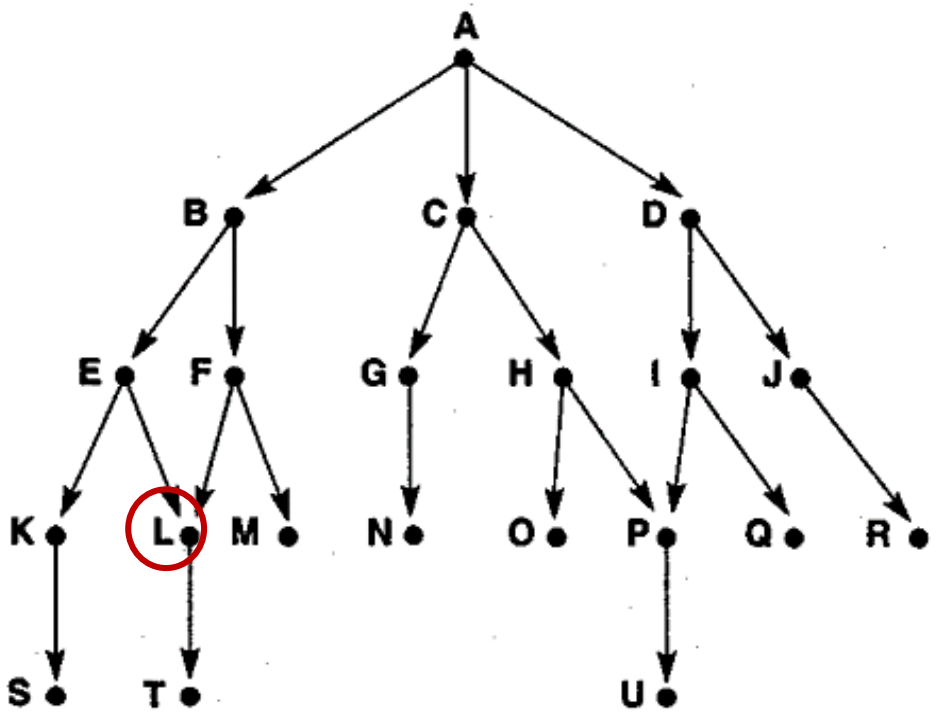
- **Memory:**

د. حسن المعيدي

Depth Limited Search (DLS) - Example

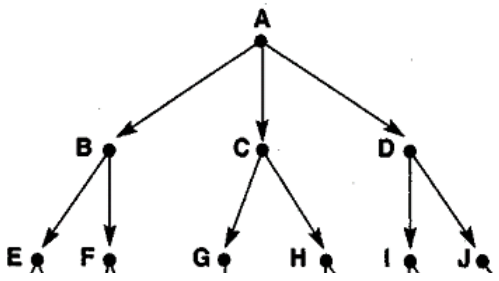
EXAMPLE

Apply the DLS algorithm for the following graph, assume the depth-limit (f) is 2.



Depth Limited Search (DLS) - Example

i	[OPEN]	X
1		
2		
3		
4		
5		
6		
7		
8		
9		
10		
11		



Depth-Limit f = 2

د. حسن المعيدي

Iterative Deepening Search (IDS)

Iterative Deepening Search (IDS)

- **Repeatedly stop DFS at a fixed depth *f* (i.e., DLS), AND**
  - **If goal is not found,**
  - **then RESTART from start node again, with f = f+1**

- **Complete Algorithm**
  - **Goal will be found when f = d, goal depth**

- **Time:**

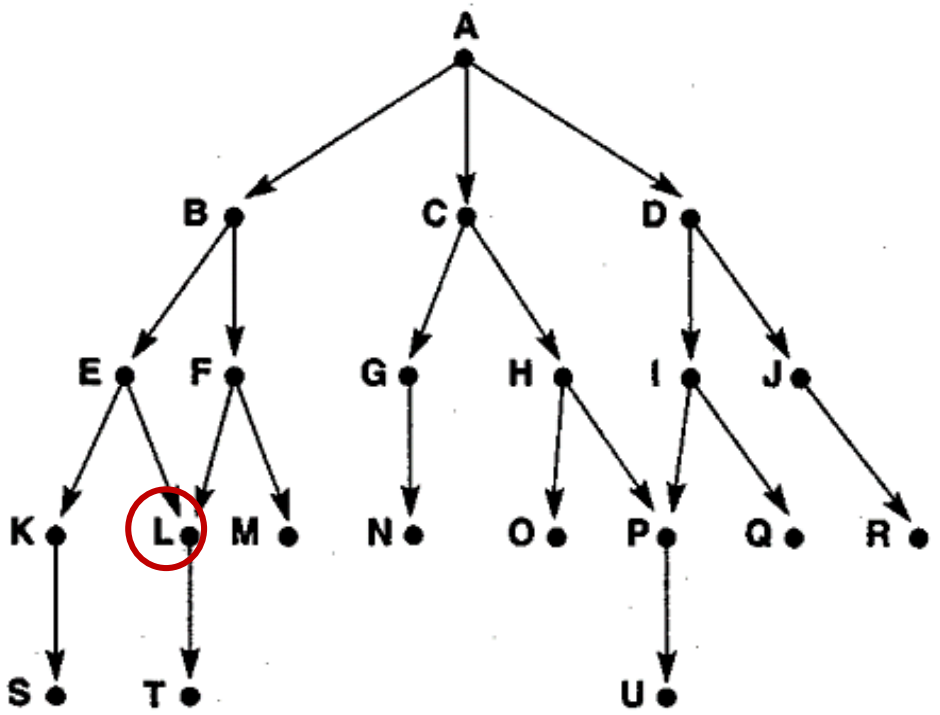
- **Memory:**

د. حسن المعيدي

Iterative Deepening Search (IDS) - Example

EXAMPLE

Apply the IDS algorithm for the following graph, start from f =0.



Iterative Deepening Search (IDS) - Example

f = 0

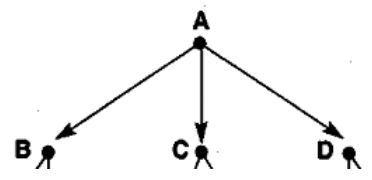
i	[OPEN]	X
1		

د. حسن المعيدي

Iterative Deepening Search (IDS) - Example

f = 1

i	[OPEN]	X
1		
2		
3		
4		

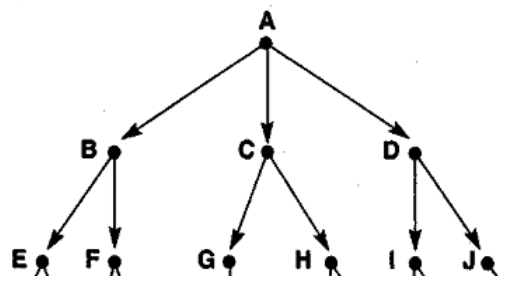


د. حسن المعيدي

Iterative Deepening Search (IDS) - Example

f = 2

i	[OPEN]	X
1		
2		
3		
4		
5		
6		
7		
8		
9		
10		

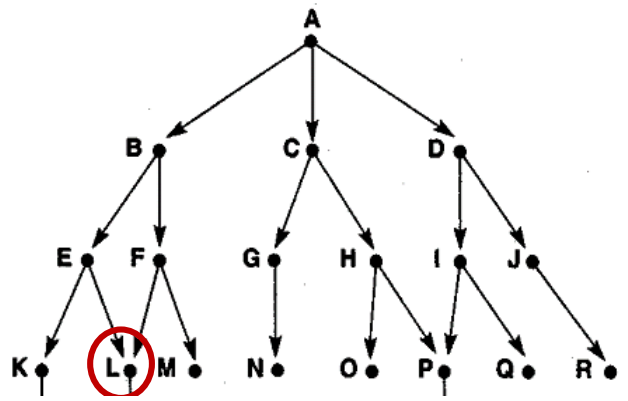


د. حسن المعيدي

Iterative Deepening Search (IDS) - Example

f = 3

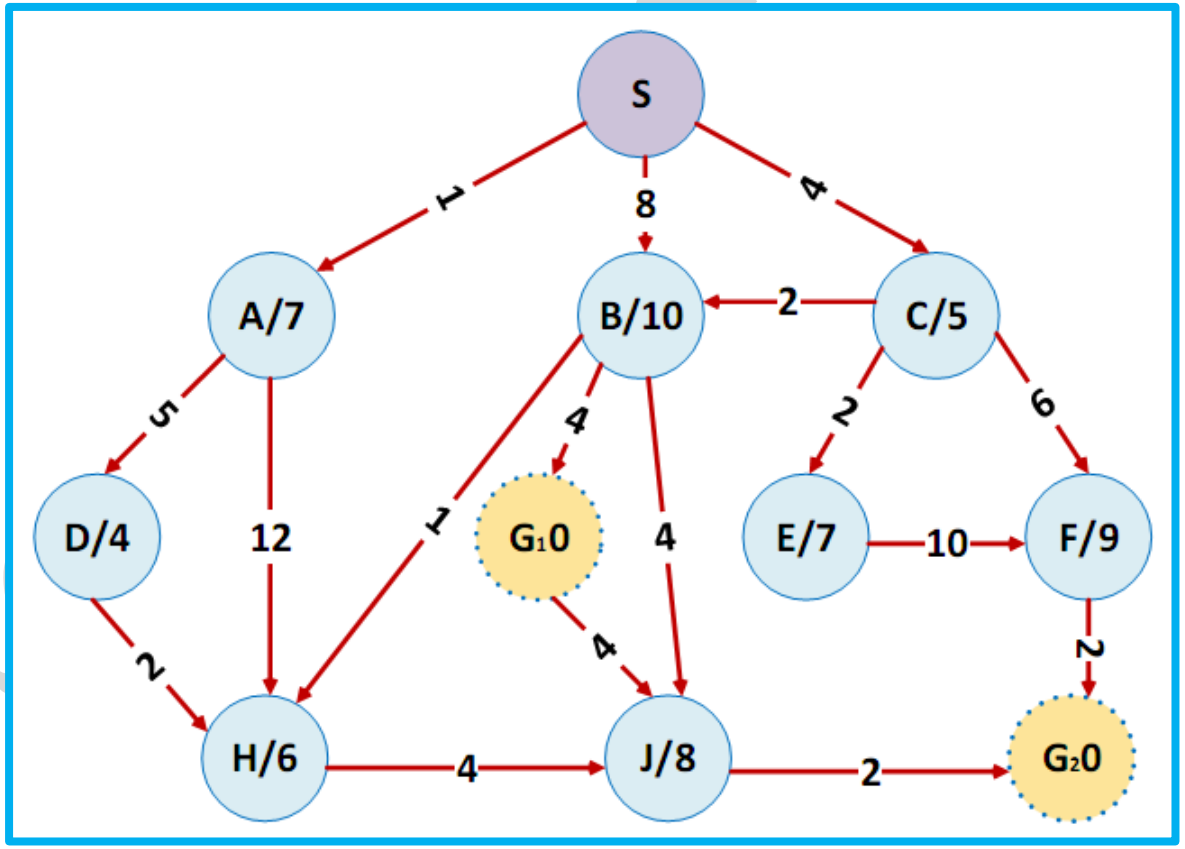
i	[OPEN]	X
1		
2		
3		
4		
5		
6		
7		
8		
9		
10		
11		
12		



د. حسن المعيدي

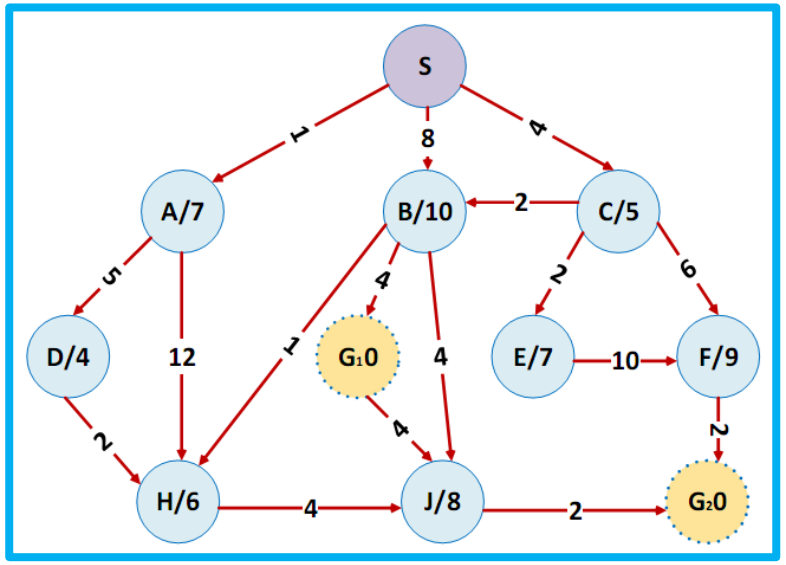
QUIZ

Apply BFS, DFS, DLS and IDS algorithms for the following graph, assume the depth limit (f) is 3.



Quiz Solution

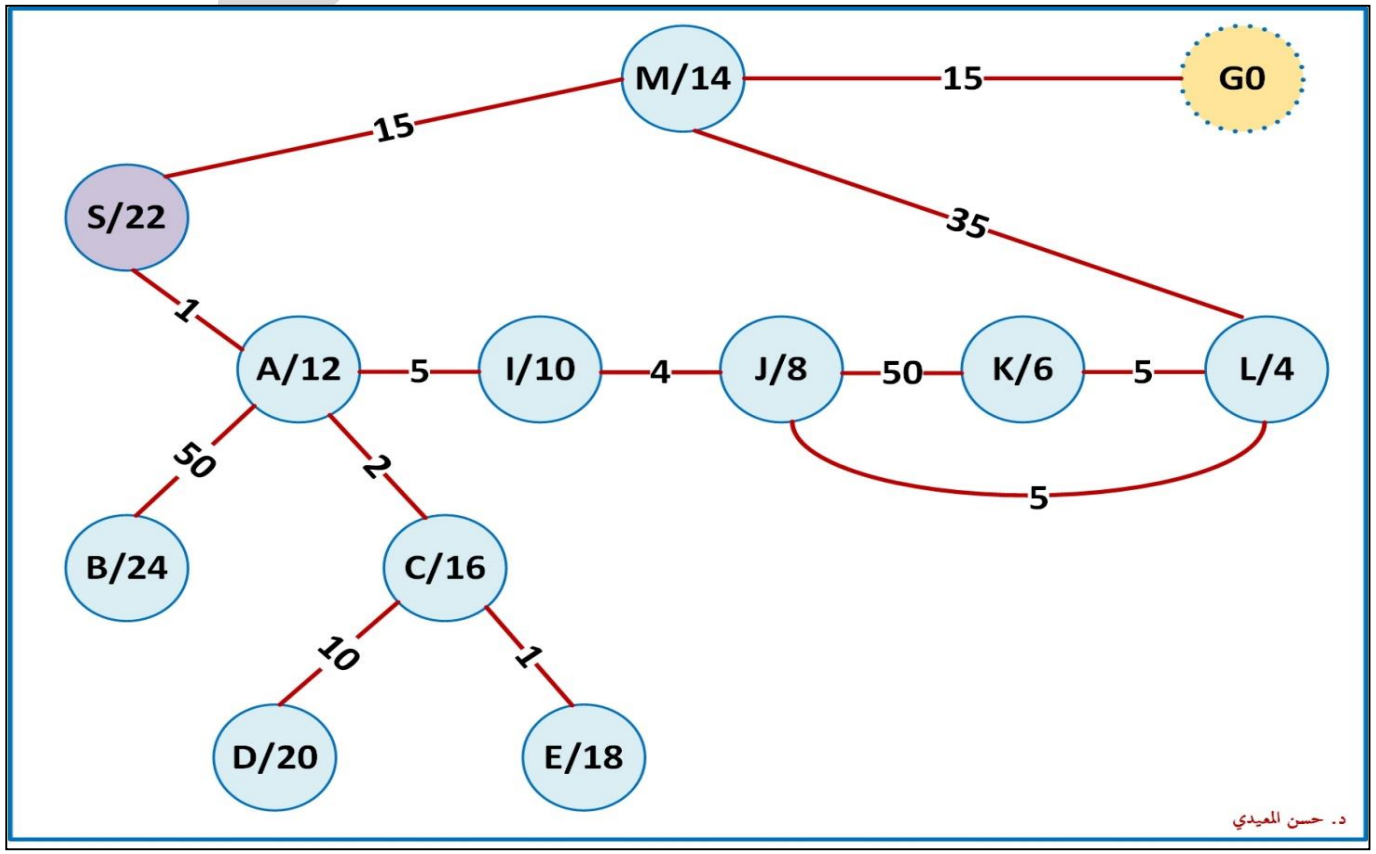
i	[OPEN]	X
1		
2		
3		
4		
5		
6		
7		
8		
9		
10		
11		



د. حسن المعيدي

QUIZ

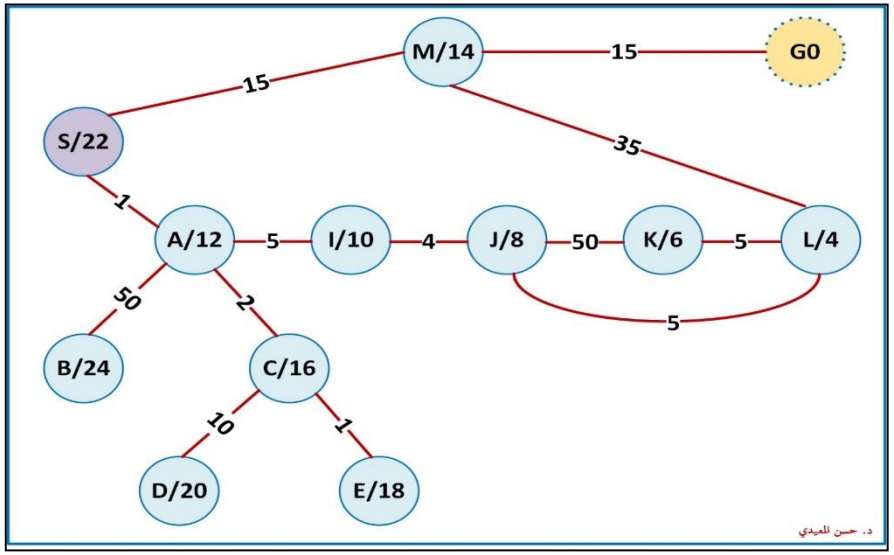
Apply BFS, DFS, DLS and IDS algorithms for the following graph, assume the depth limit (f) is 3.



المعيدي

Quiz Solution

i	[OPEN]	X
1		
2		
3		
4		
5		
6		
7		
8		
9		
10		
11		



د. حسن المعيدي

Check - Chapter Learning Outcomes

CLO

5

- Define a search problem.
- Compute the size of a search tree.
- Compute the branch factor of a search tree.
- Know the branching factors for some games.
- Compute the number of nodes in a search tree.
- Distinguish between maximum depth and goal depth.
- Approximate a time to search for a goal in a search tree.
- Approximate a space when searching for a goal in a search tree.
- Know different search strategies.
- Understand the criteria for evaluating search strategies.
- Understand, apply and evaluate (BFS, DFS, DLS, and IDS) algorithms.

End of Chapter

5